

Umami API Key — Technical Reference

Overview

API keys let you authenticate against the Umami REST API without a session cookie or user/password login. Each key is tied to a specific user account and inherits that account’s permissions. Keys are generated in the dashboard, shown once in plaintext, and then stored only as a SHA-256 hash — the server never has access to the raw key after creation.

Database Schema

The `api_key` table (Prisma model `ApiKey`) stores:

Column	Type	Notes
<code>api_key_id</code>	UUID	Primary key
<code>user_id</code>	UUID	Foreign key → user table
<code>name</code>	VARCHAR(100)	Human-readable label
<code>key_hash</code>	VARCHAR(64)	SHA-256 hex digest of the raw key (unique)
<code>key_prefix</code>	VARCHAR(20)	First 14 chars of raw key (display only)
<code>created_at</code>	TIMESTAMPTZ	Auto-set on insert
<code>revoked_at</code>	TIMESTAMPTZ	NULL = active; non-NULL = soft-deleted

A key is considered valid if and only if `revoked_at` IS NULL.

Key Format

Every generated key follows this structure:

`umami_<64 hex characters>`

Example:

`umami_a3f9c1b2d4e5f6071819202122232425262728292a2b2c2d2e2f303132333`

- The `umami_` prefix is hardcoded and is what the auth layer uses to distinguish API keys from JWT session tokens.

- The 64-character suffix is `crypto.randomBytes(32).toString('hex')` — 256 bits of entropy.
- Total length: 70 characters.

The stored `key_prefix` is the first 14 characters (`umami_` + 8 hex chars) and is used purely for identification in the UI.

Generating a Key

Via the Dashboard (recommended)

1. Log in to the Umami instance.
2. Go to **Settings** → **Profile**.
3. Scroll to the **API keys** section.
4. Click **Generate API key**.
5. Enter a descriptive name (1–100 characters) and click **Generate**.
6. **Copy the key immediately**. It is displayed once and never retrievable again — only its hash is persisted.

Via the API

You must be authenticated with an existing session token or API key to create a new key.

```
curl -X POST https://<your-umami-host>/api/me/api-keys \
  -H "Authorization: Bearer <existing-session-token-or-api-key>" \
  -H "Content-Type: application/json" \
  -d '{"name": "iOS app production"}'
```

Response (200 OK):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "iOS app production",
  "keyPrefix": "umami_a3f9c1b2",
  "createdAt": "2026-05-22T10:00:00.000Z",
  "key":
    "umami_a3f9c1b2d4e5f6071819202122232425262728292a2b2c2d2e2f303132333435"
}
```

The key field is present **only in this response**. All subsequent GET `/api/me/api-keys` responses return `keyPrefix` but never `key`.

Validation error (400 Bad Request) if name is missing or out of range:

```
{
  "error": {
    "message": "Bad request",
    "code": "bad-request",
    "status": 400
  }
}
```

```
}  
}
```

Listing Keys

```
curl https://<your-umami-host>/api/me/api-keys \  
-H "Authorization: Bearer <api-key>"
```

Response (200 OK):

```
[  
  {  
    "id": "550e8400-e29b-41d4-a716-446655440000",  
    "name": "iOS app production",  
    "keyPrefix": "umami_a3f9c1b2",  
    "createdAt": "2026-05-22T10:00:00.000Z"  
  }  
]
```

Returns all non-revoked keys for the authenticated user, newest first. The raw key value is never returned.

Revoking a Key

```
curl -X DELETE https://<your-umami-host>/api/me/api-keys/<key-id>  
\  
-H "Authorization: Bearer <api-key>"
```

Response (200 OK):

```
{ "ok": true }
```

Revocation is a soft-delete: the row remains in the database with `revoked_at` set to the current timestamp. The key becomes invalid immediately — any subsequent request using it will return 401.

If the key ID does not exist or belongs to a different user:

```
{ "error": { "message": "Not found", "code": "not-found",  
            "status": 404 } }
```

Using a Key to Authenticate Requests

Header Schema

Authorization: Bearer <full-api-key>

This is the standard HTTP Authorization header with the Bearer scheme. There is no custom header (X-Umami-API-Key does not exist in this codebase).

```
curl https://<your-umami-host>/api/websites \
  -H "Authorization: Bearer
      umami_a3f9c1b2d4e5f6071819202122232425262728292a2b2c2d2e2f303132333435"
```

Swift Implementation

```
var request = URLRequest(url: url)
request.setValue("Bearer \($apiKey)", forHTTPHeaderField:
    "Authorization")
request.setValue("application/json", forHTTPHeaderField: "Content-
    Type")
```

Server-Side Authentication Flow

When a request arrives with an Authorization: Bearer <token> header, the server executes the following in src/lib/auth.ts:

1. Extract the bearer token from the Authorization header.
 2. Check whether the token starts with umami_.
 - **Yes (API key path):** Compute SHA-256(token) → look up key_hash in the api_key table where revoked_at IS NULL. If found, load the associated user.
 - **No (JWT session path):** Attempt to parse as a signed JWT. If valid, load the user from userId claim.
 3. If no user resolves from either path, checkAuth returns null.
 4. parseRequest in src/lib/request.ts calls checkAuth and, if it returns null, sets the response to unauthorized().
-

Error Responses

Scenario	Status	Body
No Authorization header	401	{"error": {"message": "Unauthorized", "code": "unauthorized", "status": 401}}
Token present but key not found in DB	401	same as above
Key found but revoked_at is set	401	same as above
Invalid JSON body on POST	400	{"error": {"message": "Bad request", "code": "bad-request", "status": 400, ...}}
name missing or empty string	400	same as above, with Zod validation details

Scenario	Status	Body
Key ID not found on DELETE	404	<code>{"error":{"message":"Not found","code":"not-found","status":404}}</code>

There is no 403 Forbidden in the API key path — the server treats an unresolvable token the same as a missing one and returns 401.

Security Properties

- **Zero plaintext retention.** The raw key is generated, returned once, and never stored. The database row holds only the SHA-256 digest.
- **No brute-force surface.** 256 bits of entropy (32 random bytes) makes guessing computationally infeasible.
- **Immediate revocation.** Soft-delete sets `revoked_at`; the `getApiKeyByHash` query filters `revokedAt: null`, so the key is rejected on the next request after revocation.
- **Scope = issuing user.** A key carries exactly the permissions of the user who created it — no separate role or scope field exists on the key record.

Converting This File to PDF on macOS

WeasyPrint is installed (`/opt/homebrew/bin/weasyprint`) and `pandoc` is available. Run from the repository root:

```
pandoc umami_apikey_instructions.md \  
  --pdf-engine=weasyprint \  
  -o umami_apikey_instructions.pdf
```

Alternatively, open the file in any browser and use **File** → **Print** → **Save as PDF** (enable “Background graphics” for proper table rendering).